ІНСТРУМЕНТИ ТА ПРИНЦИПИ ВЕБ-ТЕХНОЛОГІЙ · ЛЕКЦІЯ

Продуктивність веб-застосунків

Метрики → як браузер виконує код → інструменти → 5 живих демо → як довести виправлення

~80 хв · далі Лаб. 7: аудит продуктивності

ЧОМУ ЦЕ ВИМОГА

Повільна сторінка коштує користувачів і грошей

+32 %

ймовірність відмови, коли завантаження сповільнюється з 1 до 3 с

Google, 2017

+8 %

продажів у Vodafone після покращення LCP на 31 %

web.dev, кейс 2021

4–6×

настільки бюджетний Android повільніший за ноутбук розробника

ваш ноутбук не репрезентативний

Також: Core Web Vitals — сигнал ранжування Google. А повільність накопичується непомітно: кожен коміт додає трохи.

СПРИЙНЯТТЯ

Що таке «швидко» для людини

16,7 мс
бюджет кадру при 60 FPS; для JS реально ~10 мс

50 мс
межа long task: запас, щоб відповісти на ввід за 100 мс

100 мс
реакція відчувається миттєвою

1 с
користувач ще тримає фокус

10 с+
користувач іде

Модель RAIL: **R**esponse · **A**nimation · **I**dle · **L**oad

МЕТРИКИ

Core Web Vitals: три метрики

Метрика	Що вимірює	Добре	Погано
LCP — Largest Contentful Paint	Коли намальовано найбільший елемент першого екрана (завантаження)	≤ 2,5 с	> 4 с
INP — Interaction to Next Paint	Затримка від взаємодії до наступного кадру (чуйність)	≤ 200 мс	> 500 мс
CLS — Cumulative Layout Shift	Наскільки «стрибає» верстка (візуальна стабільність)	≤ 0,1	> 0,25

- Оцінюють **75-й перцентиль** реальних відвідувань, окремо для mobile і desktop.
- INP замінив FID у березні 2024: FID міряв лише затримку початку першої взаємодії.
- Допоміжні метрики: TTFB, FCP, TBT (лабораторний «замінник» INP).

LCP ПІД МІКРОСКОПОМ

LCP — це сума чотирьох фаз

1

TTFB

Сервер віддав перший байт HTML

2

Load delay

Скільки браузер чекав, перш ніж почати вантажити ресурс

3

Load duration

Саме завантаження ресурсу

4

Render delay

Ресурс є, але кадр не малюється: CSS, JS, шрифти

```
new PerformanceObserver((list) ⇒ {
  const last = list.getEntries().at(-1);
  console.log('LCP', Math.round(last.startTime), last.element);
}).observe({ type: 'largest-contentful-paint', buffered: true });
```

Кандидати: <img>, <image> у SVG, постер <video>, background-image, блок тексту

INP ПІД МІКРОСКОПОМ

Кожна взаємодія складається з трьох частин

Input delay	Processing time	Presentation delay
Main thread зайнятий іншою задачею — подія чекає в черзі	Виконуються ваші обробники подій	React-рендер, style, layout, paint до наступного кадру

- INP сторінки ≈ найгірша взаємодія за весь візит (без рідкісних викидів).
- «Повільна кнопка» ≠ «повільний обробник»: input delay — наслідок чужої long task.
- Будь-яка частина може бути винуватцем — трейс показує, яка саме.

CLS КОРОТКО

Чи «стрибає» верстка

Layout shift — видимий елемент змінив позицію без дії користувача.

CLS — найгірше «вікно сесії» зсувів: до 5 с, паузи менше 1 с.

Зсуви протягом 500 мс після взаємодії користувача не рахуються.

Типові категорії причин

- медіа без зарезервованого місця
- банери й реклама, вставлені над контентом
- вебшрифти з іншими метриками
- анімації не через transform

Де шукати: доріжка Layout shifts у Performance panel, список елементів, що зсувались, у Lighthouse.

ДЖЕРЕЛА ДАНИХ

Лабораторні та польові дані

	Лабораторні (lab)	Польові (field, RUM)
Джерела	Lighthouse, Performance panel, WebPageTest	CrUX, PageSpeed Insights, web-vitals у вашому коді
Умови	Контрольовані, відтворювані	Реальні пристрої, мережі, поведінка
INP	Немає (заміна — TBT)	Є
Для чого	Дебаг, порівняння «до/після»	Чи є проблема в реальних користувачів

```
import { onLCP, onINP, onCLS } from 'web-vitals';
const send = (m) ⇒ navigator.sendBeacon('/analytics', JSON.stringify({ name: m.name, value: m.value }));
onLCP(send); onINP(send); onCLS(send);
```

MAIN THREAD I EVENT LOOP

Один потік на все — і long tasks його блокують

Main thread виконує JS, обробники подій, React-рендер і style → layout → paint.

Цикл: **одна** macrotask → **усі** microtasks → можливо, рендеринг → наступна задача.

Задача понад 50 мс — **long task**: ні кліків, ні кадрів.

TBT = Σ (тривалість − 50 мс) усіх long tasks між FCP і TTI.

```
button.addEventListener('click', async () => {
  for (let i = 0; i < 1e6; i++) {
    await Promise.resolve();
  }
});
// кадр НЕ намалюється до кінця циклу
```

Microtasks не звільняють потік

Щоб віддати потік, потрібна нова macrotask (setTimeout, MessageChannel) або scheduler.yield().

ІНСТРУМЕНТИ

Карта: симптом → інструмент

Симптом	Перший інструмент
«Довго відкривається перша сторінка»	Lighthouse → Performance (LCP by phase)
«Інтерфейс підвисає, кнопки гальмують»	Performance panel (Main, Interactions)
«Що довше працює, то повільніше; вкладка їсть RAM»	Memory panel (heap snapshots)
«Дані з’являються довго»	Network panel (waterfall)
«React рендерить забагато»	React DevTools Profiler

Інструмент обираємо за симптомом, а не за підозрою в коді.

PERFORMANCE PANEL

Чесний запис: умови важать не менше за цифри

Умови

Інкогніто без розширень; production-збірка для чесних цифр, dev — лише для React Profiler.

Capture settings

CPU: 4× або 6× slowdown · Network: Fast 4G або Slow 4G · Screenshots · за потреби Memory.

Два режими запису

Record — дія користувача. Record and reload — завантаження сторінки.

Live metrics

На стартовому екрані панелі: локальні LCP, CLS, INP одразу під час взаємодії.

Записуйте коротко, 3–5 с: довгий трейс важко читати. 4× ≈ смартфон середнього класу, 6× ≈ бюджетний.

ЧИТАННЯ ТРЕЙСУ

Як читати flame chart

Що на екрані

- По горизонталі — час, по вертикалі — стек викликів.
- Сірий «Task» з червоним трикутником і штрихуванням — long task.
- Жовтий — JS, фіолетовий — style і layout, зелений — paint.
- Широкий блок унизу стеку — власний час функції; угорі — лише обгортка.

Де шукати відповідь

- **Bottom-Up** — де витрачено self time.
- Summary, Call Tree, Event Log — для виділеної ділянки.
- Доріжки Timings, Interactions, Network, Frames.
- **Insights** — підказки, які треба перевірити трейсом.

СЕРЦЕ ЛЕКЦІЇ І ЛАБИ

Виміряй → гіпотеза → виправ → переміряй

1

Базова лінія

Сценарій, фіксовані умови, медіана з 3–5 запусків

2

Гіпотеза

«X повільно, бо Y; якщо Z — метрика зміниться на N»

3

Виправ

Одну зміну за раз

4

Переміряй

У тих самих умовах, порівняй з базовою лінією

5

Зафіксуй

Цифри, трейс і тест, що не дасть регресії

Пастки: оптимізація «на око» · dev проти prod · один запуск · кілька змін одразу · «оптимізація» коду, що займає 0,1 % часу.

ДЕМО 1 · 6 ХВ Вкладка «1. Квартальний звіт»

Long task блокує main thread

СИМПТОМ

Звіт із 600 000 рядків: годинник завмирає, кнопка «Позначити переглянутим» не реагує.

ГІПОТЕЗА

Уся генерація — одна синхронна задача на main thread.

ПЕРЕВІРКА

Performance panel, CPU 4×: доріжка Interactions і Bottom-Up.

```
const generate = () ⇒ {
  // ~0,5 с без throttling, ~2 с при CPU 4×
  const report = buildSalesReport({ rows: 600_000 });
  setReport(report);
};
```

≈ 480 мс
одна long task

≈ 490 мс
найдовший кадр

ЯК РОЗБИТИ ДОВГУ ЗАДАЧУ

Розбиття не прискорює роботу — воно повертає чуйність

Підхід	Main thread	Ціна
Синхронно	Блокований ~0,5 с	Інтерфейс не реагує
Частинами + scheduler.yield()	Задачі ≤ ~10 мс, кліки обробляються	Кадри рідше, ~10 FPS
Частинами + setTimeout(0)	Плавні кадри	Загальний час +50 %, ~0,75 с
Web Worker	Вільний	Немає DOM; дані копіюються (structured clone)

```
async function runInChunks(builder) {
  let start = performance.now();
  while (builder.process(500)) {
    if (performance.now() - start > 8) {
      await scheduler.yield();
      start = performance.now();
    }
  }
}
```

```
// fallback для yield:
// new Promise(r ⇒ setTimeout(r, 0))

const worker = new Worker(
  new URL('./reportWorker.js', import.meta.url),
  { type: 'module' },
);
worker.postMessage({ rows: 600_000 });
```

ПАМ’ЯТЬ: GC І ВИТОКИ В SPA

Витік — непотрібний об’єкт, на який досі є посилання

GC прибирає лише **недосяжні** об’єкти.

Класика SPA: таймери й підписки без відписки, глобальні listeners, кеші без меж, замикання з великими даними, detached DOM.

Memory panel: Heap snapshot, Allocations on timeline, Allocation sampling.

Три знімки: базовий → дія N разів → ще раз; Comparison, # Delta, Retained Size, Retainers.

```
useEffect(() ⇒ {
  const id = setInterval(tick, 1000);
  window.addEventListener('resize', onResize);

  // cleanup: при unmount і перед
  // повторним запуском ефекту
  return () ⇒ {
    clearInterval(id);
    window.removeEventListener('resize',
      onResize);
  };
}, []);
```

ДЕМО 2 · 6 ХВ Вкладка «2. Живі сповіщення»

Витік пам’яті: три heap snapshots

СИМПТОМ

Віджет вантажить 20 000 подій і підписується на таймер і resize. Після багатьох перемикань пам’ять росте.

ГІПОТЕЗА

Після unmount таймер живий і тримає масив подій.

ПЕРЕВІРКА

3 heap snapshots → Comparison → фільтр FeedEvent → Retainers.

9 → 59 МБ
heap після 20 монтувань; після виправлення — назад до базового

21 → 1
активних підписок

+400 000
FeedEvent, # Delta

`FeedEvent ← Array ← recent (замикання) ← колбек таймера ← таймер`

ДЕМО 3 · 4 ХВ Вкладка «3. Завантаження даних» · справжня залежність одна: orders потребує user.id

Мережевий waterfall → Promise.all

```
// Було: ~1,8 с — сума всіх затримок
const user = await getJson('/api/user');
const orders = await getJson(`/api/orders?userId=${user.id}`);
const products = await getJson('/api/products');
const stats = await getJson('/api/stats');
const notifications = await getJson('/api/notifications');
```

```
// Стало: ~0,8 с — найдовший ланцюжок user → orders
const userWithOrders = getJson('/api/user').then(async (user) => ({
  user, orders: await getJson(`/api/orders?userId=${user.id}`),
}));
const [{ user, orders }, products, stats, notifications] = await Promise.all([
  userWithOrders, getJson('/api/products'), getJson('/api/stats'), getJson('/api/notifications'),
]);
```

REACT DEVTOOLS

React Profiler: скільки рендериться і чому

Вкладка **Profiler** → запис → дія → стоп.

Commit — одне застосування змін до DOM; смужки вгорі — коміти, висота — тривалість.

Flamegraph — дерево компонентів у коміті; **Ranked** — найдорожчі першими.

Налаштування → **Highlight updates when components render** — підсвічування наживо.

НАЛАШТУВАННЯ → RECORD WHY EACH COMPONENT RENDERED

«Why did this render?»

- props changed
- state changed
- **context changed** — побачимо в демо 4
- parent rendered

```
<Profiler id onRender={…}>
```

ДЕМО 4 · 5 ХВ Вкладка «4. Ціни й валюта»

Нестабільне value у Context Provider

СЦЕНА

Таблиця з 300 товарів, ціни форматуються з контексту валюти. У Provider годинник «Оновлено: …» тікає щосекунди.

СИМПТОМ У PROFILER

Щосекунди коміт, у якому рендеряться всі 300 ProductRow. Причина — «Context changed».

ОЧІКУВАНО (DEV)

≈ 1 коміт/с, 15–20 мс; після виправлення — жодного коміту таблиці.

```
// Проблема: новий об’єкт на кожен рендер
const value = {
  currency, setCurrency,
  format: (a) ⇒ formatPrice(a, currency),
};

// Виправлення: той самий об’єкт,
// доки не змінилась валюта
const value = useMemo(() ⇒ ({
  currency, setCurrency,
  format: (a) ⇒ formatPrice(a, currency),
}), [currency]);
```

LIGHTHOUSE

Що він рахує і чого не рахує

Score 0–100 — зважена сума метрик:



Що дає

- Simulated throttling (за замовчуванням) або DevTools throttling — реальне сповільнення
- Insights: render-blocking requests, LCP breakdown, network dependency tree
- Режими: Navigation, Timespan, Snapshot

Обмеження

- Один запуск ≠ істина: шум ±5–10 балів
- Немає INP — лише TBT
- Navigation міряє тільки завантаження сторінки
- Score — не мета; мета — метрики

ДЕМО 5 · 5 ХВ index.html проти index-fixed.html · порт 4173

Render-blocking ресурси та шрифти ⟶ LCP

LCP-елемент — заголовок банера h1.hero-title; проблема у фазі **render delay**.

```
<!— Було —>
<link rel="stylesheet" href="/vendor/theme.css">
<!— усередині: @import fonts.css —>
<script src="/vendor/analytics.js"></script>

<!— Стало —>
<link rel="preload" as="font" type="font/woff2" crossorigin
  href="/vendor/fonts/inter-cyrillic-800-normal.woff2">
<style>/* critical CSS + font-display: swap */</style>
<script src="/vendor/analytics.js" defer></script>
```

LCP, Lighthouse mobile

3,9 ⟶ 1,2 с

FCP

2,9 ⟶ 1,2 с

TBT зріс: роботу перенесли, а не прибрали

0 ⟶ 120–370 мс

ТАК ВИГЛЯДАЄ ЗДАЧА ЛАБ. 7

Як довести, що виправлення працює

1. **Сценарій і умови:** prod-збірка, throttling, інкогніто, браузер.
2. **Цифри до/після:** медіана з ≥ 3 запусків.
3. **Скріншоти трейсу** до і після, той самий масштаб.
4. **Причина** одним реченням: «бо X відбувалося на main thread…»
5. **Тест-запобіжник** від регресії ↓

Метрика	До	Після
Найдовша задача, звіт (CPU 4×)	1 950 мс	0 long tasks
LCP (Lighthouse mobile, медіана з 3)	3,9 с	1,2 с

```
it('прибирає таймер і listener після unmount', () ⇒ {
  vi.useFakeTimers();
  const removeSpy = vi.spyOn(window, 'removeEventListener');
  const { unmount } = render(<LiveFeedWidget mode="fixed" />);
  unmount();
  expect(vi.getTimerCount()).toBe(0);
  expect(removeSpy).toHaveBeenCalledWith('resize', expect.any(Function));
});
```

ПІДСУМОК

Що забираємо на Лаб. 7

Метрики

LCP — завантаження, INP — чуйність, CLS — стабільність; 75-й перцентиль.

Main thread один

Long task понад 50 мс — погана чуйність.

Інструмент за симптомом

Performance, Memory, Network, React Profiler, Lighthouse.

Метод

Базова лінія → гіпотеза → одна зміна → переміряти → задокументувати.

Лаб. 7: інший проєкт, інші причини — ті самі інструменти й метод. Не все, що виглядає повільним у коді, повільне насправді.

web.dev/vitals · developer.chrome.com/docs/devtools/performance · react.dev/reference/react/Profiler